

1. What is IoC Container ?

IoC = **Inversion of Control**

Normally Java me hum object khud banate hain:

```
Student s = new Student();
```

But Spring me object banane ka kaam Spring karta hai.

Ye kaam jo karta hai usko bolte hain:

IoC Container

Container:

- object create karta hai
- manage karta hai
- dependencies inject karta hai

Spring managed object = **Bean**

2. Types of Containers in Spring

Spring me mainly 2 containers hote hain:

Container	Meaning
BeanFactory	Basic Container
ApplicationContext	Advanced Container

2.1 BeanFactory

Ye Spring ka basic/lightweight container hai.

BeanFactory factory;

Features

- Lazy initialization
- Bean tabhi create hoga jab use karenge

- Lightweight
- Old approach

Example

Agar 10 beans hain aur 1 use hua,
to sirf wahi create hoga.

2.2 ApplicationContext

Ye advanced container hai.

ApplicationContext context;

ya

AnnotationConfigApplicationContext context;

Features

- Automatically beans create karta hai
- Enterprise features deta hai
- Most used
- Fast development

Example

```
AnnotationConfigApplicationContext context =  
new AnnotationConfigApplicationContext(AppConfig.class);
```

Ye pura Spring container start karta hai.

Difference Between BeanFactory and ApplicationContext

BeanFactory	ApplicationContext
-------------	--------------------

Basic container	Advanced container
-----------------	--------------------

Lazy loading	Eager loading
--------------	---------------

Old	Modern
-----	--------

BeanFactory	ApplicationContext
--------------------	---------------------------

Lightweight	Powerful
-------------	----------

Less features	More features
---------------	---------------

3. Bean Kya Hai ?

Bean = Spring managed object

Example:

```
@Service
class Manager
{
}
```

Spring internally object banata hai:

```
Manager m = new Manager();
```

Ye object bean kehlata hai.

4. Important Annotations

4.1 @Component

Spring ko batata hai:

“Is class ka bean banao”

```
@Component
class Employee
{
}
```

Spring automatically object bana dega.

4.2 @Service

Special type of @Component

Business logic classes ke liye use hota hai.

```
@Service
class Manager
{
}
```

Internally ye bhi component hi hai.

4.3 @Repository

Database related classes ke liye.

```
@Repository
class UserDao
{
}
```

4.4 @Controller

Web request handle karne ke liye.

```
@Controller
class HomeController
{
}
```

4.5 @Configuration

Batata hai:

“Ye configuration class hai”

```
@Configuration
class AppConfig
{
}
```

4.6 @ComponentScan

Spring ko bolta hai:

“Kis package me beans dhundne hain”

```
@ComponentScan(basePackages="com.dependencyinjection")
```

Spring scan karega:

- @Component
- @Service
- @Repository
- @Controller

wali classes.

4.7 @Autowired

Dependency injection ke liye use hota hai.

Spring automatically object inject karta hai.

Example:

```
@Autowired
```

```
Allocator allocator;
```

Spring khud allocator ka object dega.

4.8 @Primary

Jab multiple beans same type ke ho.

Example:

```
class Manager implements Allocator
```

```
class TeamLead implements Allocator
```

Spring confuse ho jayega:

“Kisko inject karu?”

Tab use karte hain:

@Primary

@Service

class TeamLead implements Allocator

Matlab:

“Default bean ye wala hai”

5. Dependency Injection (DI)

Dependency = ek class ko dusri class chahiye.

Example:

```
class Car
{
    Engine engine;
}
```

Car depends on Engine.

Spring automatically dependency provide karta hai.

Isko bolte hain:

Dependency Injection

6. Types of Dependency Injection

3 types:

Type	Injection
Constructor DI	Constructor ke through
Setter DI	Setter method ke through
Field DI	Variable pe directly

6.1 Constructor Injection

Best and recommended approach.

```
@Service
class Manager
{
    private Allocator allocator;

    @Autowired
    public Manager(Allocator allocator)
    {
        this.allocator = allocator;
    }
}
```

Flow

- Spring constructor call karta hai
- dependency pass karta hai

Advantages

- Immutable
 - Safe
 - Recommended
-

6.2 Setter Injection

Setter method se dependency inject hoti hai.

```
@Service
class Manager
{
    private Allocator allocator;

    @Autowired
    public void setAllocator(Allocator allocator)
    {
        this.allocator = allocator;
    }
}
```

Advantages

- Optional dependency possible
- Flexible

Disadvantage

- Object incomplete ho sakta hai
-

6.3 Field Injection

Direct variable pe injection.

```
@Service
class Manager
{
    @Autowired
    private Allocator allocator;
}
```

Advantage

- Short code

Disadvantage

- Testing difficult
 - Recommended nahi hai large projects me
-

Best DI Type

Type	Recommended
Constructor DI	✅ Best
Setter DI	Good
Field DI	Less preferred

7. Complete Flow of Spring Program

1. Main method starts
 2. Container create hota hai
 3. ComponentScan package scan karta hai
 4. Beans create hote hain
 5. @Autowired dependency inject karta hai
 6. @Primary decide karta hai kaunsa bean inject hoga
 7. Bean use hota hai
-

8. Easy Real Life Analogy

Restaurant = Spring Container

Chef = Bean

Customer = Your class

Food supply = Dependency Injection

Tumhe khud sab kuch banana nahi padta.
Restaurant manage karta hai.

Exactly waise hi Spring object manage karta hai.

One Line Revision

Topic	Meaning
IoC Container	Objects manage karta hai
Bean	Spring managed object
@Component	Bean banao
@Service	Service bean
@Autowired	Dependency inject karo
@Primary	Default bean choose karo
Constructor DI	Constructor se injection

Topic	Meaning
Setter DI	Setter se injection
Field DI	Direct variable injection

Why Configuration is Needed in Spring?

Spring ko ye batana padta hai:

1. Kaunsi classes ke objects (beans) banane hain
2. Dependencies kaise inject karni hain
3. Beans ko kaise manage karna hai

Is process ko bolte hain:

Spring Configuration

Spring Me 3 Main Configuration Styles

No Configuration Style

- 1 XML Based Configuration
 - 2 Annotation Based Configuration
 - 3 Java Based Configuration
-

COMMON FLOW IN ALL CONFIGURATIONS

Har configuration style me internally same kaam hota hai:

Container Start

↓

Beans Create

↓

Dependencies Inject

↓

Beans Use

Difference sirf configuration likhne ke tareeke me hai.

Common Example Used

Humare paas:

Allocator -> Interface

Manager -> Implementation

Delegate -> Dependent class

Relationship

Delegate depends on Allocator

and

Manager implements Allocator

1. XML BASED CONFIGURATION

Theory

Is approach me:

- bean definitions
- dependency injection

sab XML file me likhte hain.

Spring XML file read karke beans create karta hai.

XML File Name

Mostly:

applicationContext.xml

Folder Structure

src/main/resources
applicationContext.xml

XML Configuration File

```
<bean id="allocator"  
      class="com.configurationstyle.Manager"/>  
  
<bean id="delegate"  
      class="com.configurationstyle.DelegateXMLBased">  
  
    <constructor-arg ref="allocator"/>  
  
</bean>
```

Theory of Above XML

Bean Creation

```
<bean id="allocator"  
      class="com.configurationstyle.Manager"/>
```

Meaning:

Spring Manager class ka object banao

Internally:

```
Manager allocator = new Manager();
```

Constructor Injection

```
<constructor-arg ref="allocator"/>
```

Meaning:

allocator bean constructor me bhejo

Internally:

```
new DelegateXMLBased(allocator)
```

Container Creation

```
ApplicationContext context =  
new ClassPathXmlApplicationContext("applicationContext.xml");
```

Meaning:

Spring container start karo
aur XML file load karo

Bean Get

```
DelegateXMLBased delegate =  
context.getBean("delegate", DelegateXMLBased.class);
```

Meaning:

Container se delegate bean lao

XML Based Flow

XML file read

↓

Beans create

↓

Dependencies inject

↓

Beans use

XML Based Advantages

- ✓ Easy to understand
 - ✓ Configuration visible hoti hai
 - ✓ Old projects me common
-

XML Based Disadvantages

- ✗ XML file bahut badi ho jati hai
 - ✗ Hard maintenance
 - ✗ More configuration code
-

Important XML Tags

Tag	Meaning
<beans>	Root container
<bean>	Bean create
id	Bean name
class	Class path
<constructor-arg>	Constructor injection
ref	Dusre bean ka reference

2. ANNOTATION BASED CONFIGURATION

Theory

Is approach me XML ki jagah annotations use hote hain.

Spring package scan karta hai aur annotated classes ka bean bana deta hai.

Main Idea

Class pe annotation lagao
Spring automatically bean create karega

Important Annotations

Annotation	Purpose
@Configuration	Config class
@ComponentScan	Package scan
@Component	Generic bean
@Service	Service bean
@Autowired	Dependency injection

Configuration Class

```
@Configuration
@ComponentScan(basePackages="com.configurationstyle")
public class AnnotationConfig {

}
```

Theory

@Configuration

Meaning:

Ye Spring configuration class hai

@ComponentScan

Meaning:

Is package me beans search karo

Spring scan karega:

- @Component
- @Service
- @Repository
- @Controller

wali classes.

Service Class

```
@Service
class Manager implements Allocator {

}
```

Meaning:

Manager class ka bean automatically banao

Dependent Class

```
@Component
public class DelegateAnnotationBased {

    private final Allocator allocator;

    @Autowired
    public DelegateAnnotationBased(Allocator allocator)
    {
        this.allocator = allocator;
    }
}
```

@Component

Meaning:

Is class ka bean banao

@Autowired

Meaning:

Dependency automatically inject karo

Internally:

new DelegateAnnotationBased(manager)

Container Creation

```
AnnotationConfigApplicationContext context =  
new AnnotationConfigApplicationContext(AnnotationConfig.class);
```

Meaning:

Annotation based Spring container start karo

Bean Get

```
DelegateAnnotationBased delegate =  
context.getBean(DelegateAnnotationBased.class);
```

Annotation Based Flow

@ComponentScan

↓

Package scan

↓

Annotated classes milti hain

↓

Beans create

↓

@Autowired injects dependency

Annotation Based Advantages

- ✓ Less code
 - ✓ Easy maintenance
 - ✓ Modern approach
 - ✓ Automatic bean creation
-

Annotation Based Disadvantages

- ✗ Beginners ko hidden configuration confusing lag sakti hai
-

3. JAVA BASED CONFIGURATION

Theory

Is approach me configuration pure Java class me likhte hain.

No XML required.

Hum manually methods ke through beans return karte hain.

Main Idea

@Bean method ka returned object
Spring bean ban jata hai

JavaConfig.java

```
@Configuration
public class JavaConfig {

    @Bean
    Allocator allocator()
    {
        return new Manager();
    }
}
```

```
@Bean
DelegateJavaBased delegate(Allocator allocator)
{
    return new DelegateJavaBased(allocator);
}
}
```

Theory

@Configuration

Meaning:

Ye configuration class hai

@Bean

Meaning:

Method ka returned object bean hai

allocator()

```
@Bean
Allocator allocator()
{
    return new Manager();
}
```

Meaning:

Manager object ko Spring bean bana do

delegate()

@Bean

DelegateJavaBased delegate(Allocator allocator)

Meaning:

allocator bean automatically inject karo

Internally:

new DelegateJavaBased(manager)

Container Creation

```
AnnotationConfigApplicationContext context =  
new AnnotationConfigApplicationContext(JavaConfig.class);
```

Bean Get

```
DelegateJavaBased delegate =  
context.getBean("delegate", DelegateJavaBased.class);
```

Java Config Flow

@Configuration class load

↓

@Bean methods execute

↓

Beans create

↓

Dependencies inject

Java Config Advantages

- ✓ Pure Java
- ✓ Type-safe
- ✓ Easy refactoring
- ✓ Modern & flexible

Java Config Disadvantages

✗ Beginner ko initially confusing lag sakta hai

FINAL DIFFERENCE

Feature	XML	Annotation	Java Config
Configuration Place	XML file	Annotations	Java class
Bean Creation	<bean>	@Component	@Bean
Dependency Injection	XML tags	@Autowired	Method parameter
XML Required	✓	✗	✗
Modern	✗	✓	✓
Readability	Medium	High	High

WHICH IS MOST USED TODAY?

Style	Usage
XML	Old/legacy projects
Annotation	Very common
Java Config	Most modern/common

MOST IMPORTANT FINAL UNDERSTANDING

XML Config:

Spring ko XML file se instructions milti hain

Annotation Config:

Spring annotations dekh kar beans banata hai

Java Config:

Spring Java methods se beans banata hai